

OpenNeuro ds005285: control-condition EEG → FPGA-friendly LSTM

This notebook is intentionally locked to **OpenNeuro ds005285**, **EEG only**, and the discovered **no-intervention/control session only**. It never reads **ds005284** or any other dataset directory. The execution order is also gated:

1. discover and validate the BIDS/EEGLAB metadata;
2. run small preprocessing, tensor, forward-pass, and optimizer sanity checks;
3. write **DS005285_LSTM_ARCHITECTURE.md** only after every sanity check passes;
4. only then preprocess all control subjects and train the final LSTM.

The binary target is the documented stimulus intensity ($32/s32 = \text{low}$, $64/s64 = \text{high}$). Trial-level 0–10 ratings are kept as a separate field when an authoritative rating column exists. The local v1.0.0 release is explicitly audited rather than assigning meanings to undocumented columns.

0. Environment setup

Google Colab already supplies PyTorch and most scientific packages. This cell installs only missing dependencies. Select a GPU runtime in Colab before running all cells.

```
In [1]: import importlib.util
import os
import subprocess
import sys

os.environ.setdefault("MPLCONFIGDIR", "/tmp/ds005285_matplotlib")

REQUIRED = {
    "numpy": "numpy>=1.26",
    "pandas": "pandas>=2.0",
    "scipy": "scipy>=1.11",
    "mne": "mne>=1.6",
    "torch": "torch>=2.1",
    "sklearn": "scikit-learn>=1.3",
    "matplotlib": "matplotlib>=3.7",
}

missing = [package for module, package in REQUIRED.items()
            if importlib.util.find_spec(module) is None]
if missing:
    print("Installing missing packages:", missing)
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", *missing])
else:
    print("All required packages are available.")
```

All required packages are available.

1. Imports, deterministic configuration, and ds005285 root lock

The root resolver accepts only a directory whose `dataset_description.json` declares DOI `10.18112/openneuro.ds005285...`. This is the central protection against cross-dataset use. Set environment variable `DS005285_ROOT` only if the default workspace layout differs.

```
In [2]: import copy
import hashlib
import io
import json
import math
import os
import random
import re
import time
import warnings
from collections import Counter, defaultdict
from pathlib import Path

import matplotlib.pyplot as plt
import mne
import numpy as np
import pandas as pd
import scipy
import torch
import torch.nn as nn
from scipy.io import loadmat
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    confusion_matrix,
    f1_score,
)
from torch.nn.utils import clip_grad_norm_
from torch.utils.data import DataLoader, TensorDataset

SEED = 20250722

def seed_everything(seed=SEED):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    try:
        torch.use_deterministic_algorithms(True, warn_only=True)
```

```

except TypeError:
    torch.use_deterministic_algorithms(True)

seed_everything()
mne.set_log_level("WARNING")

DATASET_DOI_PREFIX = "doi:10.18112/openneuro.ds005285"
DATASET_DIRNAME = "ds005285-download"

def _valid_ds005285(path):
    path = Path(path).expanduser().resolve()
    description = path / "dataset_description.json"
    if not description.is_file():
        return False
    metadata = json.loads(description.read_text(encoding="utf-8"))
    doi = str(metadata.get("DatasetDOI", "")).lower()
    return "ds005285" in path.name.lower() and doi.startswith(DATASET_DOI_PREFIX)

def resolve_dataset_root():
    candidates = []
    if os.environ.get("DS005285_ROOT"):
        candidates.append(Path(os.environ["DS005285_ROOT"]))
    cwd = Path.cwd().resolve()
    candidates.extend([
        cwd / "datasets" / DATASET_DIRNAME,
        cwd.parent / "datasets" / DATASET_DIRNAME,
        cwd / DATASET_DIRNAME,
        Path("/content/neuro_feedback/datasets") / DATASET_DIRNAME,
    ])
    for candidate in candidates:
        if _valid_ds005285(candidate):
            return candidate.resolve()
    raise FileNotFoundError(
        "Could not locate the local ds005285 root. Set DS005285_ROOT to the "
        "ds005285-download folder; no other dataset is accepted."
    )

DATASET_ROOT = resolve_dataset_root()
REPO_ROOT = DATASET_ROOT.parents[1]
OUTPUT_DIR = REPO_ROOT / "outputs" / "ds005285_lstm"
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
CACHE_DIR = OUTPUT_DIR / "preprocessed_cache"
CACHE_DIR.mkdir(parents=True, exist_ok=True)
ARCHITECTURE_PATH = REPO_ROOT / "DS005285_LSTM_ARCHITECTURE.md"

CONFIG = {
    "dataset": "ds005285",
    "dataset_doi": DATASET_DOI_PREFIX,
    "modality": "eeg",
    "task": "29ByANT",
    "control_condition": "SIT",
    "requested_channels": ["Fz", "Cz", "C3", "C4"],
    "line_frequency_hz": 50.0,
    "bandpass_hz": [1.0, 45.0],
    "target_sfreq_hz": 250.0,
    "epoch_s": [-0.5, 1.0],

```

```

"baseline_s": [-0.5, 0.0],
"bands_hz": {
    "delta": [1.0, 4.0],
    "theta": [4.0, 8.0],
    "alpha": [8.0, 13.0],
    "beta": [13.0, 30.0],
    "gamma": [30.0, 45.0],
},
"feature_window_s": 1.0,
"feature_stride_s": 0.25,
"feature_overlap_fraction": 0.75,
"multitaper_bandwidth_hz": 4.0,
"max_peak_to_peak_v": 500e-6,
"min_peak_to_peak_v": 0.05e-6,
"max_rejection_fraction": 0.25,
"input_size": 20,
"hidden_size": 16,
"num_layers": 1,
"num_classes": 2,
"batch_size": 64,
"learning_rate": 1e-3,
"weight_decay": 1e-4,
"max_epochs": 100,
"early_stopping_patience": 12,
"early_stopping_min_delta": 1e-4,
"gradient_clip_norm": 1.0,
"class_weight_ratio_trigger": 1.20,
"seed": SEED,
}
CONFIG_HASH = hashlib.sha256(
    json.dumps(CONFIG, sort_keys=True).encode("utf-8")
).hexdigest()[:12]
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Locked dataset root:", DATASET_ROOT)
print("Outputs:", OUTPUT_DIR)
print("Device:", DEVICE)
print("Versions:", {
    "mne": mne.__version__, "scipy": scipy.__version__,
    "numpy": np.__version__, "pandas": pd.__version__,
    "torch": torch.__version__,
})

```

```

Locked dataset root: /home/vamshi/IIT Mandi Academic Folder/HARDWARE_PROJECT
S/neuro_feedback/datasets/ds005285-download
Outputs: /home/vamshi/IIT Mandi Academic Folder/HARDWARE_PROJECTS/neuro_feed
back/outputs/ds005285_lstm
Device: cpu
Versions: {'mne': '1.12.1', 'scipy': '1.18.0', 'numpy': '2.5.1', 'pandas':
'3.0.3', 'torch': '2.13.0+cu130'}

```

2. Dataset and control-session discovery from metadata

The BIDS release has no `sessions.tsv`. Therefore the mapping is read from the embedded EEGLAB `comments` provenance (for example `.../pre/SIT/...`) and checked across every participant. `SIT` is selected only after confirming that the other three sessions are explicitly named interventions (`VR`, `VR_cTENS`, `VR_sTENS`).

```
In [3]: STIMULUS_MAP = {32: 0, 64: 1}
LABEL_NAMES = {0: "low", 1: "high"}
RATING_HINTS = ("pain_rating", "rating", "pain_score", "nrs", "vas")

def bids_paths(set_path):
    set_path = Path(set_path)
    stem = set_path.name.replace("_eeg.set", "")
    return {
        "set": set_path,
        "events": set_path.with_name(stem + "_events.tsv"),
        "channels": set_path.with_name(stem + "_channels.tsv"),
        "eeg_json": set_path.with_name(stem + "_eeg.json"),
    }

def embedded_condition(set_path):
    metadata = loadmat(
        set_path, squeeze_me=True, struct_as_record=False,
        variable_names=["comments"],
    )
    comment = str(metadata.get("comments", ""))
    match = re.search(r"[\//]pre[\//](^[^\\//]+)", comment, flags=re.IGNORECASE)
    return (match.group(1) if match else None), comment

def numeric_stimulus_code(value):
    token = str(value).strip().lower()
    match = re.fullmatch(r"s?(32|64)(?:\.\d+)?", token)
    return int(match.group(1)) if match else None

def rating_columns(events):
    found = []
    for column in events.columns:
        normalized = re.sub(r"^[a-z0-9]+", "_", column.lower()).strip("_")
        if any(hint == normalized or hint in normalized for hint in RATING_HINTS):
            found.append(column)
    return found

def audit_dataset():
    description = json.loads(
        (DATASET_ROOT / "dataset_description.json").read_text(encoding="utf-8")
    )
    event_dictionary = json.loads(
        (DATASET_ROOT / "task-29ByANT_events.json").read_text(encoding="utf-8")
    )
    records = []
    set_files = sorted(DATASET_ROOT.glob("sub-*/ses-*/eeg/*_eeg.set"))
    if not set_files:
        raise RuntimeError("No raw BIDS EEGLAB recordings found in ds005285.")
    for set_path in set_files:
        paths = bids_paths(set_path)
```

```

if not all(paths[key].is_file() for key in ("events", "channels", "e
    raise FileNotFoundError(f"Incomplete BIDS sidecars for {set_path
subject, session = set_path.name.split("_")[:2]
condition, provenance = embedded_condition(set_path)
events = pd.read_csv(paths["events"], sep="\t")
channels = pd.read_csv(paths["channels"], sep="\t")
eeg_json = json.loads(paths["eeg_json"].read_text(encoding="utf-8"))
codes = [numeric_stimulus_code(v) for v in events.get("trial_type",
records.append({
    "subject": subject,
    "session": session,
    "condition": condition,
    "set_path": str(set_path),
    "sampling_frequency": float(eeg_json["SamplingFrequency"]),
    "power_line_frequency": float(eeg_json["PowerLineFrequency"]),
    "event_columns": list(events.columns),
    "rating_columns": rating_columns(events),
    "n_stimulus_events": int(sum(code in STIMULUS_MAP for code in co
    "channels": channels["name"].astype(str).tolist(),
    "provenance_comment": provenance,
})
audit = pd.DataFrame(records)
session_conditions = (
    audit.groupby("session")["condition"]
    .agg(lambda values: sorted(set(values)))
    .to_dict()
)
expected = {
    "ses-1": ["SIT"], "ses-2": ["VR"],
    "ses-3": ["VR_cTENS"], "ses-4": ["VR_sTENS"],
}
if session_conditions != expected:
    raise RuntimeError(
        "The observed session mapping differs from the embedded ds005285
        f"{session_conditions}. Refusing to guess the control session."
    )
control = audit[(audit.session == "ses-1") & (audit.condition == "SIT")]
if len(control) != audit.subject.nunique():
    raise RuntimeError("Not every participant has exactly one ses-1/SIT
if set(audit.event_columns.map(tuple)) != {"onset", "duration", "sample
    print("NOTE: nonstandard/updated event schemas detected; rating disc
return description, event_dictionary, audit, control

```

```

DESCRIPTION, EVENT_DICTIONARY, DATASET_AUDIT, CONTROL_RECORDINGS = audit_dat
CONTROL_SESSION = "ses-1"
SUBJECTS = sorted(CONTROL_RECORDINGS.subject.unique())

```

```

print("Dataset:", DESCRIPTION["Name"], DESCRIPTION["DatasetDOI"])
print("Participants:", len(SUBJECTS))
print("Observed session → condition mapping:")
print(DATASET_AUDIT.groupby(["session", "condition"]).size())
print("Selected control/no-intervention session:", CONTROL_SESSION, "(SIT)")
print("Sampling rates:", sorted(DATASET_AUDIT.sampling_frequency.unique()))
print("Power-line frequencies:", sorted(DATASET_AUDIT.power_line_frequency.u
print("Stimulus-event counts:", DATASET_AUDIT.n_stimulus_events.value_counts
print("BIDS event schemas:", Counter(map(tuple, DATASET_AUDIT.event_columns)

```

```

print("Rating columns found:", sorted({c for cols in DATASET_AUDIT.rating_co
print("Event dictionary low/high levels:", EVENT_DICTIONARY.get("trial_type"

audit_to_save = DATASET_AUDIT.drop(columns=["channels"])
audit_to_save.to_json(OUTPUT_DIR / "dataset_audit.json", orient="records", i

```

Dataset: 29 By ANT doi:10.18112/openneuro.ds005285.v1.0.0

Participants: 29

Observed session → condition mapping:

session	condition	
ses-1	SIT	29
ses-2	VR	29
ses-3	VR_cTENS	29
ses-4	VR_sTENS	29

dtype: int64

Selected control/no-intervention session: ses-1 (SIT)

Sampling rates: [np.float64(1000.0)]

Power-line frequencies: [np.float64(50.0)]

Stimulus-event counts: {40: 116}

BIDS event schemas: Counter({'onset', 'duration', 'sample', 'value'): 116})

Rating columns found: []

Event dictionary low/high levels: {'s32': 'Laser stimli of Low intensity',
's64': 'Laser stimli of High intensity'}

3. Channel selection, event/rating extraction, and preprocessing

Exact requested channels are preferred. If one is absent, the closest unique standard 10–20/10–05 position is selected by Euclidean distance using MNE's `standard_1020` montage, and the substitution is logged. Corruption rejection is explicit and bounded: non-finite, essentially flat, or >500 μ V peak-to-peak trials are removed, with every reason retained. The pipeline aborts if more than 25% would be rejected.

```

In [4]: def choose_channels(available_names, requested=CONFIG["requested_channels"])
    available_names = list(map(str, available_names))
    lower_to_actual = {name.lower(): name for name in available_names}
    montage = mne.channels.make_standard_montage("standard_1020")
    positions = montage.get_positions()["ch_pos"]
    pos_lower = {name.lower(): np.asarray(pos) for name, pos in positions.items()}
    selected, substitutions, used = [], {}, set()
    for target in requested:
        exact = lower_to_actual.get(target.lower())
        if exact is not None and exact not in used:
            selected.append(exact)
            used.add(exact)
            substitutions[target] = exact
            continue
        if target.lower() not in pos_lower:
            raise RuntimeError(f"No standard montage position for requested {target}")
        candidates = [
            actual for actual in available_names
            if actual not in used and actual.lower() in pos_lower
        ]

```

```

        if not candidates:
            raise RuntimeError(f"No standard equivalent available for {target}")
        replacement = min(
            candidates,
            key=lambda name: np.linalg.norm(
                pos_lower[name.lower()] - pos_lower[target.lower()]
            ),
        )
        selected.append(replacement)
        used.add(replacement)
        substitutions[target] = replacement
    if len(selected) != 4 or len(set(selected)) != 4:
        raise AssertionError("Channel selection must produce four unique channels")
    return selected, substitutions


def extract_stimulus_table(events_path):
    events = pd.read_csv(events_path, sep="\t")
    original_event_columns = list(events.columns)
    source_column = "trial_type" if "trial_type" in events else "value" if "value" in events else None
    if source_column is None:
        raise RuntimeError(f"No documented stimulus column in {events_path}")
    events = events.copy()
    events["stimulus_code"] = events[source_column].map(numeric_stimulus_code_map).copy().reset_index(drop=True)
    trials = events[events.stimulus_code.isin(STIMULUS_MAP)].copy().reset_index(drop=True)
    trials["stimulus_code"] = trials.stimulus_code.astype(int)
    trials["label"] = trials.stimulus_code.map(STIMULUS_MAP).astype(int)
    candidates = rating_columns(events)
    if len(candidates) > 1:
        raise RuntimeError(
            f"Multiple possible rating columns {candidates} in {events_path}. "
            "manual metadata confirmation is required."
        )
    if candidates:
        rating_source = candidates[0]
        ratings = pd.to_numeric(trials[rating_source], errors="coerce")
        invalid = ratings.notna() & ~ratings.between(0, 10)
        if invalid.any():
            warnings.warn(
                f"{invalid.sum()} out-of-range 0-10 ratings in {events_path}"
            )
            ratings[invalid] = np.nan
        trials["subjective_rating_0_10"] = ratings.astype(float)
    else:
        rating_source = None
        trials["subjective_rating_0_10"] = np.nan
    if len(trials) == 0:
        raise RuntimeError(f"No documented 32/64 pain stimulus events in {events_path}")
    return trials, rating_source, original_event_columns


def rejection_mask(epoch_data):
    finite = np.isfinite(epoch_data).all(axis=(1, 2))
    peak_to_peak = np.ptp(epoch_data, axis=2)
    flat = (peak_to_peak < CONFIG["min_peak_to_peak_v"]).any(axis=1)
    extreme = (peak_to_peak > CONFIG["max_peak_to_peak_v"]).any(axis=1)
    keep = finite & ~flat & ~extreme
    reasons = []

```



```

for index in range(len(epoch_data)):
    trial_reasons = []
    if not finite[index]:
        trial_reasons.append("non_finite")
    if flat[index]:
        trial_reasons.append("flat_channel")
    if extreme[index]:
        trial_reasons.append("peak_to_peak_gt_500uV")
    reasons.append(";".join(trial_reasons) if trial_reasons else "kept")
return keep, reasons, peak_to_peak

def preprocess_subject(subject, limit_trials=None, verbose=True):
    row = CONTROL_RECORDINGS.loc[CONTROL_RECORDINGS.subject == subject]
    if len(row) != 1:
        raise RuntimeError(f"Expected exactly one control recording for {subject}")
    paths = bids_paths(Path(row.iloc[0].set_path))
    trials, rating_source, event_columns = extract_stimulus_table(paths["events"])
    if limit_trials is not None:
        trials = trials.iloc[: int(limit_trials)].copy()

    raw = mne.io.read_raw_eeglab(paths["set"], preload=True, verbose="ERROR")
    original_sfreq = float(raw.info["sfreq"])
    selected, substitutions = choose_channels(raw.ch_names)
    raw.pick(selected)
    raw.notch_filter(
        freqs=[CONFIG["line_frequency_hz"]],
        picks="eeg", method="fir", phase="zero", verbose="ERROR",
    )
    raw.filter(
        l_freq=CONFIG["bandpass_hz"][0],
        h_freq=CONFIG["bandpass_hz"][1],
        picks="eeg", method="fir", phase="zero", verbose="ERROR",
    )
    raw.resample(CONFIG["target_sfreq_hz"], npad="auto", verbose="ERROR")

    sfreq = float(raw.info["sfreq"])
    event_samples = np rint(trials["onset"].to_numpy(float) * sfreq).astype(int)
    mne_events = np.column_stack([
        event_samples + raw.first_samp,
        np.zeros(len(trials), dtype=int),
        trials["label"].to_numpy(int) + 1,
    ])
    metadata = trials[[
        "event_row", "onset", "stimulus_code", "label", "subjective_rating_0
    ]].copy()
    metadata.insert(0, "subject", subject)
    metadata.insert(1, "session", CONTROL_SESSION)
    metadata.insert(2, "condition", "SIT")
    metadata["trial_id"] = [f"{subject}_{CONTROL_SESSION}_event-{i:03d}" for i in range(len(metadata))]

    epochs = mne.Epochs(
        raw,
        mne_events,
        event_id={"low": 1, "high": 2},
        tmin=CONFIG["epoch_s"][0],
        tmax=CONFIG["epoch_s"][1],
    )

```

```

baseline=tuple(CONFIG["baseline_s"]),
picks=selected,
preload=True,
metadata=metadata,
reject_by_annotation=True,
event_repeated="error",
verbose="ERROR",
)
data = epochs.get_data(copy=True)
kept_metadata = epochs.metadata.reset_index(drop=True).copy()
keep, reasons, peak_to_peak = rejection_mask(data)
kept_metadata["rejection_reason"] = reasons
kept_metadata["max_peak_to_peak_uV"] = peak_to_peak.max(axis=1) * 1e6
rejection_log = kept_metadata.copy()
rejection_log["kept"] = keep
rejection_fraction = float((~keep).mean()) if len(keep) else 1.0
if rejection_fraction > CONFIG["max_rejection_fraction"]:
    raise RuntimeError(
        f"{subject}: would reject {rejection_fraction:.1%}, above the ex
        f"{CONFIG['max_rejection_fraction']:.0%} safety limit. Review is
    )
data = data[keep]
kept_metadata = kept_metadata.loc[keep].reset_index(drop=True)
del epochs, raw

report = {
    "subject": subject,
    "session": CONTROL_SESSION,
    "condition": "SIT",
    "original_sfreq_hz": original_sfreq,
    "final_sfreq_hz": sfreq,
    "available_channels": row.iloc[0].channels,
    "selected_channels": selected,
    "channel_mapping": substitutions,
    "event_columns": event_columns,
    "rating_source": rating_source,
    "ratings_available": bool(kept_metadata.subjective_rating_0_10.notna),
    "n_stimulus_trials_requested": int(len(trials)),
    "n_epochs_created": int(len(keep)),
    "n_epochs_kept": int(keep.sum()),
    "n_epochs_rejected": int((~keep).sum()),
    "rejection_fraction": rejection_fraction,
    "epoch_samples": int(data.shape[-1]) if len(data) else 0,
    "epoch_time_bounds_s": CONFIG["epoch_s"],
    "baseline_s": CONFIG["baseline_s"],
}
}
if verbose:
    print(json.dumps(report, indent=2, default=str))
    print("First trial metadata:")
    print(kept_metadata.head())
return data.astype(np.float32), kept_metadata, rejection_log, report

```

4. Sanity check A — load two participants and preprocess a few trials

This prints the sampling rate, channels/substitutions, events, labels, rating availability, epoch dimensions, and explicit rejection counts before feature extraction.

```
In [5]: SANITY_SUBJECTS = SUBJECTS[:2]
sanity_epochs = []
sanity_metadata = []
sanity_reports = []
sanity_rejection_logs = []

for subject in SANITY_SUBJECTS:
    epoch_data, trial_metadata, rejection_log, report = preprocess_subject(
        subject, limit_trials=8, verbose=True
    )
    sanity_epochs.append(epoch_data)
    sanity_metadata.append(trial_metadata)
    sanity_reports.append(report)
    sanity_rejection_logs.append(rejection_log)

sanity_epoch_tensor = np.concatenate(sanity_epochs, axis=0)
sanity_trial_metadata = pd.concat(sanity_metadata, ignore_index=True)
print("Combined sanity epoch tensor [trial, channel, sample]:", sanity_epoch_tensor)
print("Labels:", sanity_trial_metadata.label.value_counts().sort_index().to_dict())
print("Ratings present:", int(sanity_trial_metadata.subjective_rating_0_10.sum()))
```

```

{
  "subject": "sub-001",
  "session": "ses-1",
  "condition": "SIT",
  "original_sfreq_hz": 1000.0,
  "final_sfreq_hz": 250.0,
  "available_channels": [
    "Fp1",
    "Fpz",
    "Fp2",
    "F7",
    "F3",
    "Fz",
    "F4",
    "F8",
    "FC5",
    "FC1",
    "FC2",
    "FC6",
    "M1",
    "T7",
    "C3",
    "Cz",
    "C4",
    "T8",
    "M2",
    "CP5",
    "CP1",
    "CP2",
    "CP6",
    "P7",
    "P3",
    "Pz",
    "P4",
    "P8",
    "P0z",
    "O1",
    "Oz",
    "O2"
  ],
  "selected_channels": [
    "Fz",
    "Cz",
    "C3",
    "C4"
  ],
  "channel_mapping": {
    "Fz": "Fz",
    "Cz": "Cz",
    "C3": "C3",
    "C4": "C4"
  },
  "event_columns": [
    "onset",
    "duration",
    "sample",

```

```

    "value"
  ],
  "rating_source": null,
  "ratings_available": false,
  "n_stimulus_trials_requested": 8,
  "n_epochs_created": 8,
  "n_epochs_kept": 8,
  "n_epochs_rejected": 0,
  "rejection_fraction": 0.0,
  "epoch_samples": 376,
  "epoch_time_bounds_s": [
    -0.5,
    1.0
  ],
  "baseline_s": [
    -0.5,
    0.0
  ]
}

```

First trial metadata:

	subject	session	...	rejection_reason	max_peak_to_peak_uV
0	sub-001	ses-1	...	kept	53.736740
1	sub-001	ses-1	...	kept	128.489678
2	sub-001	ses-1	...	kept	33.718625
3	sub-001	ses-1	...	kept	38.445313
4	sub-001	ses-1	...	kept	33.835815

```

[5 rows x 11 columns]
{
  "subject": "sub-002",
  "session": "ses-1",
  "condition": "SIT",
  "original_sfreq_hz": 1000.0,
  "final_sfreq_hz": 250.0,
  "available_channels": [
    "Fp1",
    "Fpz",
    "Fp2",
    "F7",
    "F3",
    "Fz",
    "F4",
    "F8",
    "FC5",
    "FC1",
    "FC2",
    "FC6",
    "M1",
    "T7",
    "C3",
    "Cz",
    "C4",
    "T8",
    "M2",
    "CP5",
    "CP1",
  ]
}

```

```

        "CP2",
        "CP6",
        "P7",
        "P3",
        "Pz",
        "P4",
        "P8",
        "P0z",
        "01",
        "0z",
        "02"
    ],
    "selected_channels": [
        "Fz",
        "Cz",
        "C3",
        "C4"
    ],
    "channel_mapping": {
        "Fz": "Fz",
        "Cz": "Cz",
        "C3": "C3",
        "C4": "C4"
    },
    "event_columns": [
        "onset",
        "duration",
        "sample",
        "value"
    ],
    "rating_source": null,
    "ratings_available": false,
    "n_stimulus_trials_requested": 8,
    "n_epochs_created": 8,
    "n_epochs_kept": 8,
    "n_epochs_rejected": 0,
    "rejection_fraction": 0.0,
    "epoch_samples": 376,
    "epoch_time_bounds_s": [
        -0.5,
        1.0
    ],
    "baseline_s": [
        -0.5,
        0.0
    ]
}
First trial metadata:
  subject session ... rejection_reason max_peak_to_peak_uV
0 sub-002 ses-1 ... kept 19.142372
1 sub-002 ses-1 ... kept 16.471082
2 sub-002 ses-1 ... kept 20.562088
3 sub-002 ses-1 ... kept 23.462292
4 sub-002 ses-1 ... kept 20.618597

```

[5 rows x 11 columns]

Combined sanity epoch tensor [trial, channel, sample]: (16, 4, 376)
Labels: {0: 10, 1: 6}
Ratings present: 0

5. Sequential log-relative band-power contract

Each 1.5 s epoch uses 1.0 s DPSS multitaper windows at nominal 0.25 s stride (the 250 Hz grid gives 63 samples = 0.252 s and 74.8% overlap), producing **3 time steps**. A full second provides 1 Hz Fourier resolution, the minimum compatible with the 1 Hz delta boundary; 0.25–0.5 s windows would be invalid for delta power. The overlap gives temporal updates without pretending the steps are independent. For each channel/window, power in each band is divided by total 1–45 Hz power, clipped by machine epsilon, and natural-logged. Feature order is channel-major: Fz×5, Cz×5, C3×5, C4×5 (or documented equivalents).

```
In [6]: BAND_ITEMS = list(CONFIG["bands_hz"].items())

def feature_window_starts(n_samples, sfreq):
    window_samples = int(round(CONFIG["feature_window_s"] * sfreq))
    # Half-up rounding is explicit because Python's round uses ties-to-even:
    # 0.25 s × 250 Hz = 62.5 samples -> 63 samples, not 62.
    stride_samples = int(np.floor(CONFIG["feature_stride_s"] * sfreq + 0.5))
    starts = np.arange(0, n_samples - window_samples + 1, stride_samples, dtype=int)
    if len(starts) < 2:
        raise RuntimeError(
            f"Only {len(starts)} feature window(s); the LSTM contract requires at least 2"
        )
    return starts, window_samples, stride_samples

def extract_log_relative_bandpower(epoch_data, sfreq=CONFIG["target_sfreq_hz"]):
    epoch_data = np.asarray(epoch_data, dtype=np.float64)
    if epoch_data.ndim != 3 or epoch_data.shape[1] != 4:
        raise ValueError("Expected epoch tensor [trial, 4 channels, samples]")
    starts, window_samples, stride_samples = feature_window_starts(
        epoch_data.shape[-1], sfreq
    )
    all_features = []
    for start in starts:
        segment = epoch_data[:, :, start : start + window_samples]
        psd, freqs = mne.time_frequency.psd_array_multitaper(
            segment,
            sfreq=sfreq,
            fmin=CONFIG["bandpass_hz"][0],
            fmax=CONFIG["bandpass_hz"][1],
            bandwidth=CONFIG["multitaper_bandwidth_hz"],
            adaptive=True,
            low_bias=True,
            normalization="full",
            remove_dc=True,
            n_jobs=1,
            verbose="ERROR",
        )
```

```

if len(freqs) < 2:
    raise RuntimeError("PSD frequency grid is too sparse for band-power")
df = float(np.median(np.diff(freqs)))
total_power = np.sum(psd * df, axis=-1)
per_band = []
for band_index, (_, (low, high)) in enumerate(BAND_ITEMS):
    if band_index == len(BAND_ITEMS) - 1:
        mask = (freqs >= low) & (freqs <= high)
    else:
        mask = (freqs >= low) & (freqs < high)
    if mask.sum() == 0:
        raise RuntimeError(f"No PSD bins for band {low}-{high} Hz")
    band_power = np.sum(psd[..., mask] * df, axis=-1)
    relative = band_power / np.maximum(total_power, np.finfo(float).max)
    per_band.append(np.log(np.maximum(relative, np.finfo(np.float32).min)))
# [trial, channel, band] -> channel-major 20-vector
window_features = np.stack(per_band, axis=-1).reshape(len(epoch_data), -1)
all_features.append(window_features)
features = np.stack(all_features, axis=1).astype(np.float32)
info = {
    "window_samples": window_samples,
    "stride_samples": stride_samples,
    "window_s": window_samples / sfreq,
    "stride_s": stride_samples / sfreq,
    "effective_overlap_fraction": 1.0 - stride_samples / window_samples,
    "time_steps": int(len(starts)),
    "window_start_samples": starts.tolist(),
    "window_start_times_relative_stimulus_s": (starts / sfreq + CONFIG["stimulus_onset"]),
    "features_per_step": int(features.shape[-1]),
}
return features, info

```

```

sanity_X, FEATURE_INFO = extract_log_relative_bandpower(sanity_epoch_tensor)
sanity_y = sanity_trial_metadata.label.to_numpy(np.int64)
sanity_ratings = sanity_trial_metadata.subjective_rating_0_10.to_numpy(np.float32)
FEATURE_NAMES = [
    f"{channel}_{band}_log_relative_power"
    for channel in CONFIG["requested_channels"]
    for band, _ in BAND_ITEMS
]

```

```

assert sanity_X.shape == (len(sanity_y), FEATURE_INFO["time_steps"], 20)
assert len(FEATURE_NAMES) == 20
assert np.isfinite(sanity_X).all()
assert np.isin(sanity_y, [0, 1]).all()
print("FINAL TENSOR CONTRACT [trial, time_steps, features]:", sanity_X.shape)
print("FINAL NUMBER OF TIME STEPS:", FEATURE_INFO["time_steps"])
print("Feature-window details:", json.dumps(FEATURE_INFO, indent=2))
print("NaN count:", int(np.isnan(sanity_X).sum()), "Inf count:", int(np.isinf(sanity_X).sum()))
print("Ratings retained separately; finite count:", int(np.isfinite(sanity_ratings).sum()))

```



```

FINAL TENSOR CONTRACT [trial, time_steps, features]: (16, 3, 20)
FINAL NUMBER OF TIME STEPS: 3
Feature-window details: {
  "window_samples": 250,
  "stride_samples": 63,
  "window_s": 1.0,
  "stride_s": 0.252,
  "effective_overlap_fraction": 0.748,
  "time_steps": 3,
  "window_start_samples": [
    0,
    63,
    126
  ],
  "window_start_times_relative_stimulus_s": [
    -0.5,
    -0.248,
    0.00400000000000000036
  ],
  "features_per_step": 20
}
NaN count: 0 Inf count: 0
Ratings retained separately; finite count: 0

```

6. FPGA-friendly LSTM and optimization sanity gate

Architecture: one unidirectional LSTM ($20 \rightarrow 16$), final hidden state, and one $16 \rightarrow 2$ dense layer. This cell performs exactly one forward pass and one mini-batch AdamW step on a disposable model, then verifies finite loss and gradients. Full training variables do not yet exist, so accidental training before the gate is impossible.

```

In [7]: class PainLSTM(nn.Module):
    def __init__(self, input_size=20, hidden_size=16, num_classes=2):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=1,
            batch_first=True,
            bidirectional=False,
        )
        self.classifier = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        _, (hidden, _) = self.lstm(x)
        return self.classifier(hidden[-1])

def count_parameters(model):
    return {
        name: int(parameter.numel())
        for name, parameter in model.named_parameters()
    }

```

```

seed_everything()
sanity_model = PainLSTM().to(DEVICE)
batch_size = min(8, len(sanity_X))
batch_X = torch.from_numpy(sanity_X[:batch_size].copy()).to(DEVICE)
batch_y = torch.from_numpy(sanity_y[:batch_size].copy()).to(DEVICE)
sanity_optimizer = torch.optim.AdamW(
    sanity_model.parameters(), lr=CONFIG["learning_rate"],
    weight_decay=CONFIG["weight_decay"],
)
sanity_criterion = nn.CrossEntropyLoss()

sanity_model.train()
sanity_optimizer.zero_grad(set_to_none=True)
logits = sanity_model(batch_X)
loss = sanity_criterion(logits, batch_y)
loss.backward()
gradient_checks = {
    name: {
        "finite": bool(torch.isfinite(parameter.grad).all().item()),
        "norm": float(parameter.grad.norm().item()),
    }
    for name, parameter in sanity_model.named_parameters()
    if parameter.grad is not None
}
pre_clip_norm = float(clip_grad_norm_(
    sanity_model.parameters(), CONFIG["gradient_clip_norm"]
).item())
sanity_optimizer.step()

expected_counts = {
    "lstm.weight_ih_l0": 4 * 16 * 20,
    "lstm.weight_hh_l0": 4 * 16 * 16,
    "lstm.bias_ih_l0": 4 * 16,
    "lstm.bias_hh_l0": 4 * 16,
    "classifier.weight": 2 * 16,
    "classifier.bias": 2,
}
actual_counts = count_parameters(sanity_model)
sanity_conditions = {
    "two_participants_loaded": len(SANITY_SUBJECTS) == 2,
    "control_session_verified": CONTROL_SESSION == "ses-1" and set(CONTROL_F
    "requested_channels_or_documented_substitutes": all(len(r["selected_char
    "stimulus_labels_valid": bool(np.isin(sanity_y, [0, 1]).all()),
    "tensor_contract_valid": sanity_X.ndim == 3 and sanity_X.shape[1:] == (F
    "tensor_finite": bool(np.isfinite(sanity_X).all()),
    "logits_shape_valid": tuple(logits.shape) == (batch_size, 2),
    "loss_finite": bool(torch.isfinite(loss).item()),
    "gradients_exist": len(gradient_checks) == len(list(sanity_model.paramet
    "gradients_finite": all(item["finite"] for item in gradient_checks.value
    "nonzero_gradient": any(item["norm"] > 0 for item in gradient_checks.val
    "gradient_norm_finite": bool(np.isfinite(pre_clip_norm)),
    "parameter_counts_valid": actual_counts == expected_counts,
    "rejection_below_limit": all(r["rejection_fraction"] <= CONFIG["max_reje
}
SANITY_CHECKS_PASSED = all(sanity_conditions.values())

```

```

SANITY_REPORT = {
    "passed": SANITY_CHECKS_PASSED,
    "checks": sanity_conditions,
    "subjects": SANITY_SUBJECTS,
    "tensor_shape": list(sanity_X.shape),
    "time_steps": FEATURE_INFO["time_steps"],
    "loss": float(loss.item()),
    "pre_clip_gradient_norm": pre_clip_norm,
    "gradients": gradient_checks,
    "parameter_counts": actual_counts,
    "total_parameters": int(sum(actual_counts.values())),
    "rating_observation": (
        "No trial-level rating column exists in the audited local ds005285 v
        "events.tsv files; ratings are retained as NaN and are not fabricate
    ),
}
(OUTPUT_DIR / "sanity_check_report.json").write_text(
    json.dumps(SANITY_REPORT, indent=2), encoding="utf-8"
)
print(json.dumps(SANITY_REPORT, indent=2))
assert SANITY_CHECKS_PASSED, "Sanity gate failed; full preprocessing/trainin
print("SANITY GATE PASSED. Full training remains unavailable until the next

```

```

{
  "passed": true,
  "checks": {
    "two_participants_loaded": true,
    "control_session_verified": true,
    "requested_channels_or_documented_substitutes": true,
    "stimulus_labels_valid": true,
    "tensor_contract_valid": true,
    "tensor_finite": true,
    "logits_shape_valid": true,
    "loss_finite": true,
    "gradients_exist": true,
    "gradients_finite": true,
    "nonzero_gradient": true,
    "gradient_norm_finite": true,
    "parameter_counts_valid": true,
    "rejection_below_limit": true
  },
  "subjects": [
    "sub-001",
    "sub-002"
  ],
  "tensor_shape": [
    16,
    3,
    20
  ],
  "time_steps": 3,
  "loss": 0.6924582719802856,
  "pre_clip_gradient_norm": 0.42100992798805237,
  "gradients": {
    "lstm.weight_ih_l0": {
      "finite": true,
      "norm": 0.3231661915779114
    },
    "lstm.weight_hh_l0": {
      "finite": true,
      "norm": 0.03109653852880001
    },
    "lstm.bias_ih_l0": {
      "finite": true,
      "norm": 0.04012010246515274
    },
    "lstm.bias_hh_l0": {
      "finite": true,
      "norm": 0.04012010246515274
    },
    "classifier.weight": {
      "finite": true,
      "norm": 0.19336415827274323
    },
    "classifier.bias": {
      "finite": true,
      "norm": 0.17674000561237335
    }
  }
},

```

```

"parameter_counts": {
  "lstm.weight_ih_l0": 1280,
  "lstm.weight_hh_l0": 1024,
  "lstm.bias_ih_l0": 64,
  "lstm.bias_hh_l0": 64,
  "classifier.weight": 32,
  "classifier.bias": 2
},
"total_parameters": 2466,
"rating_observation": "No trial-level rating column exists in the audited
local ds005285 v1.0.0 events.tsv files; ratings are retained as NaN and are
not fabricated."
}
SANITY GATE PASSED. Full training remains unavailable until the next documen
tation cell completes.

```

7. Architecture record — created only after the sanity gate

This cell writes the requested root-level architecture document using verified runtime facts. The next cell also checks that the file exists before full preprocessing can start.

```

In [8]: assert SANITY_CHECKS_PASSED, "Architecture documentation cannot precede the

substitution_lines = []
for report in sanity_reports:
    mapping = report["channel_mapping"]
    substitution_lines.append(
        f"- `{report['subject']}`: " + ", ".join(f"{k}→{v}" for k, v in mapping.items())
    )
rating_columns_found = sorted([c for cols in DATASET_AUDIT.rating_columns for c in cols])
rating_note = (
    "No trial-level 0–10 rating column was found. All 116 BIDS events files
    contain `onset`, `duration`, `sample`, and `value`; raw EEGLAB events contain s
    `32`/`64` plus impedance markers. The separate rating field is therefor
    NaN. Participant-level `laser_low`/`laser_high` fields are calibration
    trial ratings, and are deliberately not substituted."
    if not rating_columns_found else
    f"Rating column(s) discovered and range-checked: {rating_columns_found}."
)
architecture_text = f"""# DS005285 LSTM Architecture

Generated only after the executable sanity gate passed on {time.strftime('%Y-%m-%d %H:%M:%S')}

## Dataset scope and control-session selection

- Dataset: OpenNeuro `ds005285`, DOI `{DESCRIPTION['DatasetDOI']}`, local BIDS
- Modality: raw EEG `.set`/`.fdt` recordings and their BIDS sidecars only; c
  other dataset directory are excluded.
- Participants: {len(SUBJECTS)} (`{SUBJECTS[0]}` through `{SUBJECTS[-1]}`),
- Metadata audit: 116 raw recordings = 29 participants × 4 sessions, 40 stim
- Embedded EEGLAB provenance maps `ses-1→SIT`, `ses-2→VR`, `ses-3→VR_cTENS`,
  `ses-4→VR_sTENS` for every participant. `ses-1/SIT` is the unique session
  TENS and is therefore the control/no-intervention condition. This mapping

```

runtime and the pipeline stops if it changes.

Final preprocessing and input contract

1. Load each `ses-1/SIT` EEGLAB recording with MNE and validate sidecars.
2. Select `Fz`, `Cz`, `C3`, `C4`; all are present in the audited release. If one, choose the closest unique standard-1020 coordinate and record the map.
3. Apply a zero-phase 50 Hz FIR notch, zero-phase 1–45 Hz FIR band-pass, then 1000 Hz to 250 Hz.
4. Epoch -0.5 to $+1.0$ s around each documented pain-stimulus onset and apply mean baseline correction.
5. Reject only explicit corruption: non-finite samples, a channel below 0.05 μV or a channel above 500 μV peak-to-peak. Save all reasons and abort for reprocessing if any participant's requested trials would be removed.
6. Estimate DPSS multitaper spectra in 1.0 s windows with a nominal 0.25 s stride; the explicit 63-sample stride is 0.252 s (74.8% overlap), bandwidth 4 Hz. This provides 1 Hz Fourier spacing, the shortest defensible window for a band beginning at 1 Hz; shorter windows are prohibited. Overlap supersedes resolution but does not make windows statistically independent.
7. For delta 1–4, theta 4–8, alpha 8–13, beta 13–30, and gamma 30–45 Hz, compute power / total 1–45 Hz power, clip only at floating-point epsilon, then take the mean.

Final tensor: `**[trial, {FEATURE_INFO['time_steps']} time_steps, 20 feature means]`. The `{FEATURE_INFO['time_steps']}` window starts are `{FEATURE_INFO['window_start']}` to the pain stimulus; feature order is four channel-major groups $\times$ five band groups. The original tensor was `{list(sanity_X.shape)}` with no NaN/Inf.

Channel verification:

```
{chr(10).join(substitution_lines)}
```

Labels and retained ratings

The dataset's `task-29ByANT_events.json` defines `s32` as low-intensity laser, `s64` as high-intensity laser. BIDS files store numeric `32`/`64`; labels are exactly `64` $\rightarrow$ `1` (high). Unknown event codes are never coerced.

```
{rating_note}
```

Subject-wise splitting and normalization

A seeded permutation of unique participant IDs creates approximately 60/20/20% training, validation, and held-out test subject sets. Assertions require pairwise-disjointness: no union is all participants. Every trial from one participant stays in exactly one set. No overlapping epoch can cross a split. The 20 feature means and standard deviations are computed over training-subject trials/time steps only, saved, and reused unchanged for validation and test. Test data do not control preprocessing, early stopping, normalization, or data augmentation.

LSTM equations and architecture

For time step `t`, input `x_t  $\in \mathbb{R}^{20}$` , previous hidden/cell states `h_(t-1)`, and PyTorch gate order `(i, f, g, o)`:

```
```text
```

```
i_t = sigmoid(W_ii x_t + b_ii + W_hi h_(t-1) + b_hi)
f_t = sigmoid(W_if x_t + b_if + W_hf h_(t-1) + b_hf)
g_t = tanh (W_ig x_t + b_ig + W_hg h_(t-1) + b_hg)
o_t = sigmoid(W_oi x_t + b_oi + W_ho h_(t-1) + b_ho)
```

```

o_t = sigmoid(W_io x_t + b_io + W_ho h_(t-1) + b_ho)
c_t = f_t ⊗ c_(t-1) + i_t ⊗ g_t
h_t = o_t ⊗ tanh(c_t)
logits = W_y h_T + b_y ∈ R^2
prediction = argmax(logits)
```

```

It is one unidirectional, one-layer LSTM with 16 hidden units followed by a layer. There is no attention, bidirectionality, dropout, convolution, or recurrent connections in the single layer.

Parameter counts

| Tensor | Shape | Parameters |
|---------------------|------------|------------------|
| `lstm.weight_ih_l0` | `(64, 20)` | 1,280 |
| `lstm.weight_hh_l0` | `(64, 16)` | 1,024 |
| `lstm.bias_ih_l0` | `(64,)` | 64 |
| `lstm.bias_hh_l0` | `(64,)` | 64 |
| `classifier.weight` | `(2, 16)` | 32 |
| `classifier.bias` | `(2,)` | 2 |
| **Total** | | **2,466** |

Training procedure

Cross-entropy loss, AdamW (`lr=1e-3`, `weight\_decay=1e-4`), batch size 64, 100 epochs, gradient norm clipping at 1.0, deterministic seed {SEED}, and early stopping on validation loss (patience 12, minimum improvement 1e-4). Inverse-frequency class weights are enabled only if the training-set majority/minority ratio exceeds 1.20, and are applied to training labels only. The best validation-loss state is restored before final evaluation.

Evaluation

Validation and held-out test reports save accuracy, balanced accuracy, macro average (TP/(TP+FN) for high pain), specificity (TN/(TN+FP) for low pain), and the [[TN, FP], [FN, TP]] confusion matrix.

Data-leakage protections

- Dataset root DOI/name lock; raw `ds005285` EEG only.
- Control mapping verified from every raw file before selection.
- Unique-subject split before normalization; split intersections asserted empty.
- Train-only normalization and optional train-only class weights.
- Validation only for early stopping; held-out test evaluated once after resampling.
- Cache keys include the complete preprocessing configuration hash.
- Trial IDs and subject IDs are exported for audit; no trial duplication across splits.

Assumptions and discovered limitations

- `SIT` is interpreted as seated/no-intervention control because all alternative conditions are explicitly intervention-labelled in embedded provenance. No top-level `session` or `subject` labels are present.
- {rating_note}
- The 1.5 s epoch bounds the lowest-frequency estimation. A 1.0 s window is the current choice for 1 Hz resolution; delta estimates remain noisier than higher bands.
- Filtering is non-causal zero-phase offline preprocessing. A future streaming implementation must replace it with validated causal filters and quantify the domain shift.

```
## Future fixed-point and FPGA export plan
```

1. Freeze the saved train-only normalization constants and feature ordering.
2. Calibrate per-tensor or per-row symmetric scales on training data only for recurrent weights, dense weights, cell state, and hidden state.
3. Implement sigmoid/tanh with bounded LUT or piecewise-linear approximation gate order `(i,f,g,o)` from the exported arrays.
4. Run bit-accurate Python inference against float32 test logits and select saturation/error/metric trade-offs without tuning on test labels.
5. Export golden feature sequences, intermediate gates/states, logits, and do co-simulation; verify every tensor dimension and bias addition.
6. Replace offline spectral/filter blocks with streaming equivalents only after numerical validation, latency/resource analysis, and end-to-end regression

```
"""
architecture_text = "\n".join(line.rstrip() for line in architecture_text.splitlines())
ARCHITECTURE_PATH.write_text(architecture_text, encoding="utf-8")
DOCUMENTATION_WRITTEN = ARCHITECTURE_PATH.is_file() and ARCHITECTURE_PATH.stat().st_size > 0
assert DOCUMENTATION_WRITTEN
print("Wrote:", ARCHITECTURE_PATH)
```

Wrote: /home/vamshi/IIT Mandi Academic Folder/HARDWARE_PROJECTS/neuro_feedback/DS005285_LSTM_ARCHITECTURE.md

8. Full control-cohort preprocessing — gated after documentation

Running this cell starts full preprocessing only because the sanity gate passed and the architecture file exists. Per-subject caches are configuration-hashed and contain no data from other datasets or sessions.

```
In [9]: assert SANITY_CHECKS_PASSED and DOCUMENTATION_WRITTEN
RUN_FULL_TRAINING = True
if not RUN_FULL_TRAINING:
    raise RuntimeError("Full training is disabled. Set RUN_FULL_TRAINING=True")

def subject_cache_path(subject):
    return CACHE_DIR / f"{subject}_{CONTROL_SESSION}_{CONFIG_HASH}.npz"

def load_or_create_subject_features(subject):
    cache_path = subject_cache_path(subject)
    if cache_path.is_file():
        cached = np.load(cache_path, allow_pickle=False)
        metadata = pd.read_json(io.StringIO(str(cached["metadata_json"].item())))
        report = json.loads(str(cached["report_json"].item()))
        rejection_log = pd.read_json(io.StringIO(str(cached["rejection_json"].item())))
        return cached["X"], cached["y"], cached["ratings"], metadata, rejection_log, report
    epochs, metadata, rejection_log, report = preprocess_subject(subject, verbose=True)
    X, feature_info = extract_log_relative_bandpower(epochs)
    if feature_info["time_steps"] != FEATURE_INFO["time_steps"]:
        raise RuntimeError(f"{subject}: inconsistent number of feature time steps")
    y = metadata.label.to_numpy(np.int64)
    ratings = metadata.subjective_rating_0_10.to_numpy(np.float32)
    if not np.isfinite(X).all():
```



```

        raise RuntimeError(f"{subject}: NaN/Inf in extracted features")
    np.savez_compressed(
        cache_path,
        X=X.astype(np.float32),
        y=y,
        ratings=ratings,
        metadata_json=np.array(metadata.to_json(orient="records")),
        rejection_json=np.array(rejection_log.to_json(orient="records")),
        report_json=np.array(json.dumps(report)),
        config_hash=np.array(CONFIG_HASH),
    )
    return X, y, ratings, metadata, rejection_log, report

cohort_X, cohort_y, cohort_ratings = [], [], []
cohort_metadata, cohort_rejections, cohort_reports = [], [], []
full_start = time.time()
for index, subject in enumerate(SUBJECTS, start=1):
    X_sub, y_sub, r_sub, meta_sub, reject_sub, report_sub = load_or_create_subject(subject)
    cohort_X.append(X_sub)
    cohort_y.append(y_sub)
    cohort_ratings.append(r_sub)
    cohort_metadata.append(meta_sub)
    cohort_rejections.append(reject_sub)
    cohort_reports.append(report_sub)
    print(f"[{index:02d}/{len(SUBJECTS)}] {subject}: {len(y_sub)} kept trials, {len(reject_sub['n_epochs_rejected'])} rejected")

X_all = np.concatenate(cohort_X).astype(np.float32)
y_all = np.concatenate(cohort_y).astype(np.int64)
ratings_all = np.concatenate(cohort_ratings).astype(np.float32)
metadata_all = pd.concat(cohort_metadata, ignore_index=True)
rejection_log_all = pd.concat(cohort_rejections, ignore_index=True)
subjects_all = metadata_all.subject.astype(str).to_numpy()
trial_ids_all = metadata_all.trial_id.astype(str).to_numpy()

assert X_all.shape == (len(y_all), FEATURE_INFO["time_steps"], 20)
assert np.isfinite(X_all).all() and np.isin(y_all, [0, 1]).all()
assert len(set(trial_ids_all)) == len(trial_ids_all)
print("Full tensor contract:", X_all.shape)
print("FINAL NUMBER OF TIME STEPS:", X_all.shape[1])
print("Class counts:", Counter(y_all.tolist()))
print("Finite ratings:", int(np.isfinite(ratings_all).sum()), "/", len(ratings_all))
print("Full preprocessing minutes:", round((time.time() - full_start) / 60, 2))

metadata_all.to_csv(OUTPUT_DIR / "trial_metadata.csv", index=False)
rejection_log_all.to_csv(OUTPUT_DIR / "trial_rejection_log.csv", index=False)
(OUTPUT_DIR / "preprocessing_reports.json").write_text(
    json.dumps(cohort_reports, indent=2, default=str), encoding="utf-8"
)

```

```

[01/29] sub-001: 40 kept trials, 0 rejected
[02/29] sub-002: 40 kept trials, 0 rejected
[03/29] sub-003: 40 kept trials, 0 rejected
[04/29] sub-004: 40 kept trials, 0 rejected
[05/29] sub-005: 40 kept trials, 0 rejected
[06/29] sub-006: 40 kept trials, 0 rejected
[07/29] sub-007: 40 kept trials, 0 rejected
[08/29] sub-008: 40 kept trials, 0 rejected
[09/29] sub-009: 40 kept trials, 0 rejected
[10/29] sub-010: 40 kept trials, 0 rejected
[11/29] sub-011: 40 kept trials, 0 rejected
[12/29] sub-012: 40 kept trials, 0 rejected
[13/29] sub-013: 40 kept trials, 0 rejected
[14/29] sub-014: 40 kept trials, 0 rejected
[15/29] sub-015: 40 kept trials, 0 rejected
[16/29] sub-016: 40 kept trials, 0 rejected
[17/29] sub-017: 40 kept trials, 0 rejected
[18/29] sub-018: 40 kept trials, 0 rejected
[19/29] sub-019: 39 kept trials, 1 rejected
[20/29] sub-020: 40 kept trials, 0 rejected
[21/29] sub-021: 40 kept trials, 0 rejected
[22/29] sub-022: 40 kept trials, 0 rejected
[23/29] sub-023: 40 kept trials, 0 rejected
[24/29] sub-024: 40 kept trials, 0 rejected
[25/29] sub-025: 40 kept trials, 0 rejected
[26/29] sub-026: 40 kept trials, 0 rejected
[27/29] sub-027: 40 kept trials, 0 rejected
[28/29] sub-028: 40 kept trials, 0 rejected
[29/29] sub-029: 40 kept trials, 0 rejected
Full tensor contract: (1159, 3, 20)
FINAL NUMBER OF TIME STEPS: 3
Class counts: Counter({0: 580, 1: 579})
Finite ratings: 0 / 1159
Full preprocessing minutes: 0.0

```

9. Strict subject split and train-only normalization

The split is performed on unique participant IDs before any normalization statistics are fit. Pairwise-disjointness and trial ownership are asserted and the exact lists are saved.

```

In [10]: def make_subject_splits(subjects, seed=SEED):
            unique = np.array(sorted(set(map(str, subjects))))
            if len(unique) < 5:
                raise RuntimeError("Too few participants for train/validation/test s
            rng = np.random.default_rng(seed)
            shuffled = rng.permutation(unique)
            n_test = max(1, int(round(0.20 * len(unique))))
            n_val = max(1, int(round(0.20 * len(unique))))
            test = sorted(shuffled[:n_test].tolist())
            val = sorted(shuffled[n_test:n_test + n_val].tolist())
            train = sorted(shuffled[n_test + n_val:].tolist())
            split_sets = list(map(set, [train, val, test]))
            assert not (split_sets[0] & split_sets[1])
            assert not (split_sets[0] & split_sets[2])

```

```

    assert not (split_sets[1] & split_sets[2])
    assert set.union(*split_sets) == set(unique)
    return {"train": train, "validation": val, "test": test}

SUBJECT_SPLITS = make_subject_splits(subjects_all)
split_indices = {
    name: np.flatnonzero(np.isin(subjects_all, subject_list))
    for name, subject_list in SUBJECT_SPLITS.items()
}
for split_name, indices in split_indices.items():
    observed = set(subjects_all[indices])
    assert observed == set(SUBJECT_SPLITS[split_name])
    assert set(np.unique(y_all[indices])) == {0, 1}

train_idx = split_indices["train"]
val_idx = split_indices["validation"]
test_idx = split_indices["test"]
feature_mean = X_all[train_idx].mean(axis=(0, 1), dtype=np.float64).astype(r
feature_std = X_all[train_idx].std(axis=(0, 1), dtype=np.float64).astype(np.
zero_variance = feature_std < 1e-8
if zero_variance.any():
    raise RuntimeError(f"Near-zero training variance in features: {np.array(
X_normalized = ((X_all - feature_mean[None, None, :]) /
                feature_std[None, None, :]).astype(np.float32)
assert np.isfinite(X_normalized).all()

with (OUTPUT_DIR / "subject_splits.json").open("w", encoding="utf-8") as har
    json.dump(SUBJECT_SPLITS, handle, indent=2)
np.savez(
    OUTPUT_DIR / "feature_normalization.npz",
    mean=feature_mean,
    std=feature_std,
    feature_names=np.array(FEATURE_NAMES),
    fitted_subjects=np.array(SUBJECT_SPLITS["train"]),
)
pd.DataFrame({"feature": FEATURE_NAMES, "mean": feature_mean, "std": feature
    OUTPUT_DIR / "feature_normalization.csv", index=False
)
print("Subject splits:", {k: len(v) for k, v in SUBJECT_SPLITS.items()})
print(json.dumps(SUBJECT_SPLITS, indent=2))

```

```

Subject splits: {'train': 17, 'validation': 6, 'test': 6}
{
  "train": [
    "sub-003",
    "sub-006",
    "sub-007",
    "sub-008",
    "sub-009",
    "sub-011",
    "sub-012",
    "sub-014",
    "sub-016",
    "sub-018",
    "sub-019",
    "sub-021",
    "sub-022",
    "sub-024",
    "sub-027",
    "sub-028",
    "sub-029"
  ],
  "validation": [
    "sub-001",
    "sub-002",
    "sub-004",
    "sub-010",
    "sub-020",
    "sub-026"
  ],
  "test": [
    "sub-005",
    "sub-013",
    "sub-015",
    "sub-017",
    "sub-023",
    "sub-025"
  ]
}

```

10. Full training with AdamW, clipping, optional class weights, and early stopping

Class weights activate only when the training-subject trial imbalance exceeds 1.20.

Early stopping monitors validation loss; test subjects remain untouched until the best state is restored.

```

In [11]: def make_loader(indices, shuffle):
          dataset = TensorDataset(
              torch.from_numpy(X_normalized[indices]),
              torch.from_numpy(y_all[indices]),
          )
          generator = torch.Generator().manual_seed(SEED)
          return DataLoader(

```

```

        dataset,
        batch_size=CONFIG["batch_size"],
        shuffle=shuffle,
        generator=generator if shuffle else None,
        num_workers=0,
        pin_memory=torch.cuda.is_available(),
    )

train_loader = make_loader(train_idx, shuffle=True)
val_loader = make_loader(val_idx, shuffle=False)
test_loader = make_loader(test_idx, shuffle=False)

train_counts = np.bincount(y_all[train_idx], minlength=2)
imbalance_ratio = float(train_counts.max() / train_counts.min())
if imbalance_ratio > CONFIG["class_weight_ratio_trigger"]:
    class_weights_np = len(train_idx) / (2.0 * train_counts.astype(np.float64))
    class_weights = torch.tensor(class_weights_np, dtype=torch.float32, device=device)
    print("Using train-only class weights:", class_weights_np.tolist())
else:
    class_weights_np = None
    class_weights = None
    print(f"Class ratio {imbalance_ratio:.3f}; class weights are unnecessary")

def loader_predictions(model, loader):
    model.eval()
    losses, y_true, y_pred, logits_all = [], [], [], []
    unweighted_loss = nn.CrossEntropyLoss(reduction="sum")
    with torch.no_grad():
        for X_batch, y_batch in loader:
            X_batch = X_batch.to(DEVICE, non_blocking=True)
            y_batch = y_batch.to(DEVICE, non_blocking=True)
            logits_batch = model(X_batch)
            losses.append(float(unweighted_loss(logits_batch, y_batch).item()))
            y_true.extend(y_batch.cpu().numpy().tolist())
            y_pred.extend(logits_batch.argmax(dim=1).cpu().numpy().tolist())
            logits_all.append(logits_batch.cpu().numpy())
    y_true = np.asarray(y_true, dtype=np.int64)
    y_pred = np.asarray(y_pred, dtype=np.int64)
    return {
        "loss": float(sum(losses) / len(y_true)),
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)),
        "macro_f1": float(f1_score(y_true, y_pred, average="macro", zero_division=0)),
        "y_true": y_true,
        "y_pred": y_pred,
        "logits": np.concatenate(logits_all),
    }

seed_everything()
model = PainLSTM().to(DEVICE)
criterion = nn.CrossEntropyLoss(weight=class_weights)
optimizer = torch.optim.AdamW(
    model.parameters(), lr=CONFIG["learning_rate"],
    weight_decay=CONFIG["weight_decay"],
)
history = []

```

```

best_validation_loss = math.inf
best_epoch = 0
best_state = None
epochs_without_improvement = 0

for epoch in range(1, CONFIG["max_epochs"] + 1):
    model.train()
    train_loss_sum, train_items = 0.0, 0
    gradient_norms = []
    for X_batch, y_batch in train_loader:
        X_batch = X_batch.to(DEVICE, non_blocking=True)
        y_batch = y_batch.to(DEVICE, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        logits_batch = model(X_batch)
        batch_loss = criterion(logits_batch, y_batch)
        if not torch.isfinite(batch_loss):
            raise FloatingPointError(f"Non-finite training loss at epoch {epoch}")
        batch_loss.backward()
        gradient_norm = clip_grad_norm(model.parameters(), CONFIG["gradient_clip_val"])
        if not torch.isfinite(gradient_norm):
            raise FloatingPointError(f"Non-finite gradient norm at epoch {epoch}")
        optimizer.step()
        train_loss_sum += float(batch_loss.item()) * len(y_batch)
        train_items += len(y_batch)
        gradient_norms.append(float(gradient_norm.item()))

    train_eval = loader_predictions(model, train_loader)
    val_eval = loader_predictions(model, val_loader)
    row = {
        "epoch": epoch,
        "optimization_train_loss": train_loss_sum / train_items,
        "train_loss": train_eval["loss"],
        "train_accuracy": train_eval["accuracy"],
        "train_balanced_accuracy": train_eval["balanced_accuracy"],
        "train_macro_f1": train_eval["macro_f1"],
        "validation_loss": val_eval["loss"],
        "validation_accuracy": val_eval["accuracy"],
        "validation_balanced_accuracy": val_eval["balanced_accuracy"],
        "validation_macro_f1": val_eval["macro_f1"],
        "mean_preclip_gradient_norm": float(np.mean(gradient_norms)),
    }
    history.append(row)
    print(
        f"epoch={epoch:03d} train_loss={row['train_loss']:.4f} "
        f"val_loss={row['validation_loss']:.4f} "
        f"val_bal_acc={row['validation_balanced_accuracy']:.3f}"
    )
    if val_eval["loss"] < best_validation_loss - CONFIG["early_stopping_min_delta"]:
        best_validation_loss = val_eval["loss"]
        best_epoch = epoch
        best_state = copy.deepcopy(model.state_dict())
        epochs_without_improvement = 0
    else:
        epochs_without_improvement += 1
        if epochs_without_improvement >= CONFIG["early_stopping_patience"]:
            print("Early stopping at epoch", epoch)

```

```

        break

if best_state is None:
    raise RuntimeError("Training completed without a valid best model state.")
model.load_state_dict(best_state)
history_frame = pd.DataFrame(history)
history_frame.to_csv(OUTPUT_DIR / "training_history.csv", index=False)
(OUTPUT_DIR / "training_history.json").write_text(
    history_frame.to_json(orient="records", indent=2), encoding="utf-8"
)
print("Restored best epoch:", best_epoch, "validation loss:", best_validation_loss)

```

Class ratio 1.003; class weights are unnecessary.

```

epoch=001 train_loss=0.6912 val_loss=0.6925 val_bal_acc=0.517
epoch=002 train_loss=0.6853 val_loss=0.6899 val_bal_acc=0.529
epoch=003 train_loss=0.6805 val_loss=0.6879 val_bal_acc=0.542
epoch=004 train_loss=0.6761 val_loss=0.6868 val_bal_acc=0.562
epoch=005 train_loss=0.6723 val_loss=0.6860 val_bal_acc=0.567
epoch=006 train_loss=0.6687 val_loss=0.6858 val_bal_acc=0.558
epoch=007 train_loss=0.6650 val_loss=0.6851 val_bal_acc=0.558
epoch=008 train_loss=0.6613 val_loss=0.6857 val_bal_acc=0.558
epoch=009 train_loss=0.6577 val_loss=0.6871 val_bal_acc=0.554
epoch=010 train_loss=0.6541 val_loss=0.6877 val_bal_acc=0.546
epoch=011 train_loss=0.6510 val_loss=0.6883 val_bal_acc=0.558
epoch=012 train_loss=0.6474 val_loss=0.6911 val_bal_acc=0.558
epoch=013 train_loss=0.6443 val_loss=0.6936 val_bal_acc=0.550
epoch=014 train_loss=0.6410 val_loss=0.6968 val_bal_acc=0.550
epoch=015 train_loss=0.6377 val_loss=0.6993 val_bal_acc=0.550
epoch=016 train_loss=0.6348 val_loss=0.7013 val_bal_acc=0.554
epoch=017 train_loss=0.6315 val_loss=0.7059 val_bal_acc=0.554
epoch=018 train_loss=0.6286 val_loss=0.7066 val_bal_acc=0.542
epoch=019 train_loss=0.6255 val_loss=0.7071 val_bal_acc=0.542

```

Early stopping at epoch 19

Restored best epoch: 7 validation loss: 0.6851335684458415

11. Evaluation and complete artifact export

Metrics use fixed label order `[0, 1]`, so sensitivity is high-pain recall and specificity is low-pain recall. PyTorch LSTM arrays retain gate order `(input, forget, cell, output)`.

```

In [12]: def classification_metrics(model, loader):
    result = loader_predictions(model, loader)
    cm = confusion_matrix(result["y_true"], result["y_pred"], labels=[0, 1])
    tn, fp, fn, tp = cm.ravel()
    result.update({
        "sensitivity": float(tp / (tp + fn)) if (tp + fn) else float("nan"),
        "specificity": float(tn / (tn + fp)) if (tn + fp) else float("nan"),
        "confusion_matrix": cm,
    })
    return result

validation_result = classification_metrics(model, val_loader)
test_result = classification_metrics(model, test_loader)

```

```

def serializable_metrics(result):
    return {
        key: value.tolist() if isinstance(value, np.ndarray) else value
        for key, value in result.items()
        if key not in {"y_true", "y_pred", "logits"}
    }

METRICS = {
    "best_epoch": best_epoch,
    "validation": serializable_metrics(validation_result),
    "test": serializable_metrics(test_result),
}

print(json.dumps(METRICS, indent=2))
(OUTPUT_DIR / "metrics.json").write_text(json.dumps(METRICS, indent=2), encoding="utf-8")

cm = test_result["confusion_matrix"]
np.save(OUTPUT_DIR / "confusion_matrix.npy", cm)
pd.DataFrame(cm, index=["true_low", "true_high"], columns=["pred_low", "pred_high"],
              index_label="True", column_label="Predicted",
              dtype="int",
              storage_options={"path": OUTPUT_DIR / "confusion_matrix.csv"})

fig, ax = plt.subplots(figsize=(4.5, 4.0))
image = ax.imshow(cm, cmap="Blues")
for i in range(2):
    for j in range(2):
        ax.text(j, i, int(cm[i, j]), ha="center", va="center")
ax.set(
    xticks=[0, 1], yticks=[0, 1],
    xticklabels=["low", "high"], yticklabels=["low", "high"],
    xlabel="Predicted", ylabel="True", title="ds005285 held-out test confusion matrix"
)
fig.colorbar(image, ax=ax)
fig.tight_layout()
fig.savefig(OUTPUT_DIR / "confusion_matrix.png", dpi=180)
plt.close(fig)

MODEL_CONFIG = {
    **CONFIG,
    "config_hash": CONFIG_HASH,
    "control_session": CONTROL_SESSION,
    "feature_info": FEATURE_INFO,
    "feature_names": FEATURE_NAMES,
    "time_steps": FEATURE_INFO["time_steps"],
    "total_parameters": sum(count_parameters(model).values()),
    "parameter_counts": count_parameters(model),
    "pytorch_lstm_gate_order": ["input", "forget", "cell", "output"],
    "class_weights": None if class_weights_np is None else class_weights_np,
    "best_epoch": best_epoch,
}

(OUTPUT_DIR / "model_config.json").write_text(
    json.dumps(MODEL_CONFIG, indent=2), encoding="utf-8"
)

torch.save(
    {
        "model_state_dict": {k: v.detach().cpu() for k, v in model.state_dict().items()},
        "model_config": MODEL_CONFIG,
    },
    OUTPUT_DIR / "model_state_dict.pth"
)

```



```

        "feature_mean": feature_mean,
        "feature_std": feature_std,
        "subject_splits": SUBJECT_SPLITS,
        "metrics": METRICS,
    },
    OUTPUT_DIR / "best_model.pt",
)

weights_dir = OUTPUT_DIR / "lstm_weights"
weights_dir.mkdir(exist_ok=True)
exported_arrays = {}
for name, tensor in model.state_dict().items():
    safe_name = name.replace(".", "_")
    array = tensor.detach().cpu().numpy()
    exported_arrays[safe_name] = array
    np.save(weights_dir / f"{safe_name}.npz", array)
    np.savetxt(weights_dir / f"{safe_name}.csv", array.reshape(array.shape[0]))
np.savez(weights_dir / "all_model_parameters.npz", **exported_arrays)
(weights_dir / "README.json").write_text(json.dumps({
    "gate_order": ["input", "forget", "cell", "output"],
    "note": "Rows 0:16,16:32,32:48,48:64 of each LSTM weight/bias are i,f,g,
    "arrays": {name: list(array.shape) for name, array in exported_arrays.items()}
}, indent=2), encoding="utf-8")

predictions = metadata_all.iloc[test_idx][[
    "trial_id", "subject", "session", "condition", "stimulus_code",
    "label", "subjective_rating_0_10"
]].copy()
predictions["predicted_label"] = test_result["y_pred"]
predictions[["logit_low", "logit_high"]] = test_result["logits"]
predictions.to_csv(OUTPUT_DIR / "test_predictions.csv", index=False)

required_artifacts = [
    "best_model.pt", "training_history.json", "training_history.csv",
    "confusion_matrix.npy", "confusion_matrix.csv", "confusion_matrix.png",
    "metrics.json", "feature_normalization.npz", "feature_normalization.csv",
    "subject_splits.json", "model_config.json", "trial_metadata.csv",
    "trial_rejection_log.csv", "test_predictions.csv",
    "lstm_weights/all_model_parameters.npz",
]
missing_artifacts = [name for name in required_artifacts if not (OUTPUT_DIR / name).exists()]
assert not missing_artifacts, f"Missing required artifacts: {missing_artifacts}"
print("All required artifacts saved under:", OUTPUT_DIR)
for name in required_artifacts:
    print(" -", name)

```

```

{
  "best_epoch": 7,
  "validation": {
    "loss": 0.6851335684458415,
    "accuracy": 0.5583333333333333,
    "balanced_accuracy": 0.5583333333333333,
    "macro_f1": 0.5545906576090762,
    "sensitivity": 0.4666666666666667,
    "specificity": 0.65,
    "confusion_matrix": [
      [
        78,
        42
      ],
      [
        64,
        56
      ]
    ]
  },
  "test": {
    "loss": 0.6692817529042562,
    "accuracy": 0.6,
    "balanced_accuracy": 0.6,
    "macro_f1": 0.5982142857142857,
    "sensitivity": 0.5333333333333333,
    "specificity": 0.6666666666666666,
    "confusion_matrix": [
      [
        80,
        40
      ],
      [
        56,
        64
      ]
    ]
  }
}

```

All required artifacts saved under: /home/vamshi/IIT Mandi Academic Folder/H
ARDWARE_PROJECTS/neuro_feedback/outputs/ds005285_lstm

- best_model.pt
- training_history.json
- training_history.csv
- confusion_matrix.npy
- confusion_matrix.csv
- confusion_matrix.png
- metrics.json
- feature_normalization.npz
- feature_normalization.csv
- subject_splits.json
- model_config.json
- trial_metadata.csv
- trial_rejection_log.csv
- test_predictions.csv
- lstm_weights/all_model_parameters.npz

